COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 4, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Mr. Jimlord Quejado

ACTIVITY NO. 4 Stacks

1. PROCEDURE OUTPUT

ILO B.2

Code:

```cpp
#include <iostream>

class Node {
public:
    int data;
    Node* next;
};

Node *head = NULL, *tail = NULL;

void push(int newData) {
    Node *newNode = new Node;
    newNode->data = newData;
    newNode->next = head;
    if (head == NULL) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head = newNode;
    }
}

int pop() {
    int tempVal;
    Node *temp;
    if (head == NULL) {
        head = tail = NULL;
        std::cout << "Stack Underflow." << std::endl;
        return -1;
    } else {
        temp = head;
        tempVal = temp->data;
        head = head->next;
        delete temp;
```

```
35          return tempVal;
36      }
37  }
38
39  void Top() {
40      if (head == NULL) {
41          std::cout << "Stack is Empty." << std::endl;
42          return;
43      } else {
44          std::cout << "Top of Stack: " << head->data << std::endl;
45      }
46  }
47
48
49  void display() {
50      if (head == NULL) {
51          std::cout << "Stack is Empty." << std::endl;
52          return;
53      }
54      Node* temp = head;
55      std::cout << "Stack elements (Top to Bottom): ";
56      while (temp != NULL) {
57          std::cout << temp->data << " ";
58          temp = temp->next;
59      }
60      std::cout << std::endl;
61  }
62
```

```
63  int main() {
64      push(1);
65      std::cout << "After the first PUSH, stack elements: ";
66      display();
67
68      push(5);
69      std::cout << "After the second PUSH, stack elements: ";
70      display();
71
72      pop();
73      std::cout << "After the first POP operation, stack elements: ";
74      display();
75
76      pop();
77      std::cout << "After the second POP operation, stack elements: ";
78      display();
79
80      pop();
81      return 0;
82  }
```

Output:

```
After the first PUSH, stack elements: Stack elements (Top to Bottom): 1
After the second PUSH, stack elements: Stack elements (Top to Bottom): 5 1
After the first POP operation, stack elements: Stack elements (Top to Bottom): 1
After the second POP operation, stack elements: Stack is Empty.
Stack Underflow.
```

Analysis

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary A

```cpp
#include <iostream>
#include <string>

const int MAX_SIZE = 100;

class ArrayStack {
private:
    char arr[MAX_SIZE];
    int top;

public:
    ArrayStack() {
        top = -1;
    }

    bool isEmpty() {
        return top == -1;
    }

    bool isFull() {
        return top == MAX_SIZE - 1;
    }

    void push(char val) {
        if (isFull()) {
            std::cout << "Stack Overflow\n";
            return;
        }
        arr[++top] = val;
    }

    char pop() {
        if (isEmpty()) {
            return '\0';
```

```
35          }
36          return arr[top--];
37      }
38  };
39
40  bool isMatchingPair(char open, char close) {
41      if (open == '(' && close == ')') return true;
42      if (open == '{' && close == '}') return true;
43      if (open == '[' && close == ']') return true;
44      return false;
45  }
46
47  bool checkBalancedArray(const std::string& expr) {
48      ArrayStack stack;
49
50      for (char ch : expr) {
51
52          if (ch == '(' || ch == '{' || ch == '[') {
53              stack.push(ch);
54          }
55
56          else if (ch == ')' || ch == '}' || ch == ']') {
57              if (stack.isEmpty()) {
58                  return false;
59              }
60              char topChar = stack.pop();
61              if (!isMatchingPair(topChar, ch)) {
62                  return false;
63              }
```

```
64          }
65      }
66
67      return stack.isEmpty();
68  }
69
70  int main() {
71      std::string testCases[] = {
72          "(A+B)+(C-D)",
73          "((A+B)+(C-D)",
74          "((A+B)+[C-D])",
75          "((A+B]+[C-D]}"
76      };
77
78      std::cout << "=== Array Stack Symbol Checker ===\n";
79      for (const auto& expr : testCases) {
80          bool isValid = checkBalancedArray(expr);
81          std::cout << "Expression: " << expr
82                    << " | Valid: " << (isValid ? "Y" : "N") << "\n";
83      }
84
85      return 0;
86  }
```

Supplementary B:

Code:

```cpp
#include <iostream>
#include <string>

class Node {
public:
    char data;
    Node* next;

    Node(char val) : data(val), next(nullptr) {}
};

class LinkedListStack {
private:
    Node* top;

public:
    LinkedListStack() : top(nullptr) {}

    ~LinkedListStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    bool isEmpty() {
        return top == nullptr;
    }

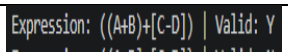
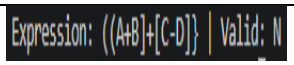
    void push(char val) {
        Node* newNode = new Node(val);
        newNode->next = top;
        top = newNode;
    }

```

```
35      char pop() {
36          if (isEmpty()) {
37              return '\0';
38          }
39          Node* temp = top;
40          char poppedValue = temp->data;
41          top = top->next;
42          delete temp;
43          return poppedValue;
44      }
45  };
46
47  bool isMatchingPairList(char open, char close) {
48      if (open == '(' && close == ')') return true;
49      if (open == '{' && close == '}') return true;
50      if (open == '[' && close == ']') return true;
51      return false;
52  }
53
54  bool checkBalancedLinkedList(const std::string& expr) {
55      LinkedListStack stack;
56
57      for (char ch : expr) {
58          if (ch == '(' || ch == '{' || ch == '[') {
59              stack.push(ch);
60          }
61          else if (ch == ')' || ch == '}' || ch == ']') {
62              if (stack.isEmpty()) {
63                  return false;
```

```
63                  return false;
64              }
65              char topChar = stack.pop();
66              if (!isMatchingPairList(topChar, ch)) {
67                  return false;
68              }
69          }
70      }
71
72      return stack.isEmpty();
73  }
74
75  int main() {
76      std::string testCases[] = {
77          "(A+B)+(C-D)",
78          "((A+B)+(C-D)",
79          "((A+B)+[C-D])",
80          "((A+B]+[C-D]}"
81      };
82
83      std::cout << "=== Linked List Stack Symbol Checker ===\n";
84      for (const auto& expr : testCases) {
85          bool isValid = checkBalancedLinkedList(expr);
86          std::cout << "Expression: " << expr
87                    << " | Valid: " << (isValid ? "Y" : "N") << "\n";
88      }
89
90      return 0;
91  }
```

Expression	Valid? (Y/N)	Output (Console Screenshot	Analysis
(A+B) + (C-D)	Y	Expression: (A+B)+(C-D) | Valid: Y	All opening parentheses (are correctly matched with closing parentheses). The stack is completely empty at the end
((A+B) + (C-D)	N	Expression: ((A+B)+(C-D) | Valid: N	Two (symbols were pushed, but only one) was processed at the end. The stack is not

			empty when reaching the end of input.
((A+B) + [C-D])	Y		Both sets of (and [their corresponding) and] in proper nested order.
((A+B] + [C-D]}	N		The first] encounters (on top of the stack, causing a symbol mismatch error.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept explanation, problem solving guidance and code verification**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used solely as a learning assistant to explain the mechanics of stacks and clarify the activity requirements. I developed, implemented and tested all the C++ code for the array, linked list and STL implementations independently. The AI did not replace my original thinking, code development or analysis.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.